

# Python to JavaScript

## Structured Learning Track for Python Developers

A phased roadmap from language fundamentals to framework-ready web development

|             |               |                                 |                             |
|-------------|---------------|---------------------------------|-----------------------------|
| 5<br>Phases | 6–10<br>Weeks | Vanilla JS<br>first, then React | Project-based<br>milestones |
|-------------|---------------|---------------------------------|-----------------------------|

## Learning Phases Overview

### PHASE 1

### Core JS Fundamentals + ES6+

~3 weeks

The deepest phase — covers the language from scratch through modern ES6+ patterns. Structured into 7 blocks following javascript.info Part 1. See the detailed breakdown on the next pages.

- Block 1: JS Fundamentals (variables, types, operators, functions)
- Block 2: Objects (methods, this, constructors, optional chaining)
- Block 3: Data Types in Depth (arrays, destructuring, Map/Set, JSON)
- Block 4: Advanced Functions (closures, spread/rest, arrow functions revisited)
- Block 5: Classes & Prototypes (inheritance, static, private fields)
- Block 6: Async JS (callbacks, Promises, async/await)
- Block 7: Modules (import/export, dynamic imports)

*Milestone: Rewrite a small Python CLI tool in JS and run it with Node.js.*

### PHASE 2

### The DOM & Browser APIs

1 week

This is where JavaScript meets web pages. Python has no equivalent — entirely new territory.

- `document.querySelector('#id')` / `document.querySelectorAll('.class')`
- Reading and writing element content: `element.textContent`, `element.innerHTML`
- Listening for events: `element.addEventListener('click', handler)`
- Creating and inserting elements dynamically: `createElement`, `appendChild`
- Fetch API for HTTP requests — replaces Python's requests library

```
fetch('https://api.example.com/data')  
.then(res => res.json()).then(data => console.log(data));
```

*Milestone: Build a live search filter — type in a box, filter a list of items in real time.*

**PHASE 3****First Real Project (Vanilla JS)**

1–2 weeks

Apply everything before touching a framework. Frameworks hide complexity you need to understand first.

- Build a To-Do app: add, complete, delete, persist to localStorage
- Or build a Weather Dashboard: fetch live data from a public API, render dynamically
- Or build a Quiz App: track state, validate answers, show a score

Focus areas: organise code into modules, handle async errors, separate UI and data logic.

*Milestone: Ship the project to GitHub Pages with a live URL.*

**PHASE 4****React (Framework Foundations)**

2–3 weeks

Once Phase 3 is done, React will make sense — not feel like magic.

- Components: reusable UI building blocks (functions that return JSX)
- Props: passing data into components (like function arguments)
- State: useState hook — local data that triggers re-renders
- Effects: useEffect hook — run code when data or props change
- Start with the official React docs at react.dev — they are excellent

*Milestone: Re-implement your Phase 3 project in React, component by component.*

## Phase 1 — Detailed Block Breakdown

All topics map directly to javascript.info Part 1. Work through them in order. Complete the exercises at the bottom of each page before moving on.

### BLOCK 1 JS Fundamentals

4–5 days

| Topic                 | javascript.info URL                                                                                           | Python Dev Note                          |
|-----------------------|---------------------------------------------------------------------------------------------------------------|------------------------------------------|
| Hello, world!         | <a href="https://javascript.info/hello-world">javascript.info/hello-world</a>                                 | Running JS in browser vs Node            |
| Code structure        | <a href="https://javascript.info/structure">javascript.info/structure</a>                                     | Semicolons, statement rules              |
| use strict            | <a href="https://javascript.info/strict-mode">javascript.info/strict-mode</a>                                 | Always enable — makes JS saner           |
| Variables             | <a href="https://javascript.info/variables">javascript.info/variables</a>                                     | let/const/var vs Python assignment       |
| Data types            | <a href="https://javascript.info/types">javascript.info/types</a>                                             | 8 types incl. null, undefined, NaN       |
| Type conversions      | <a href="https://javascript.info/type-conversions">javascript.info/type-conversions</a>                       | The == coercion rules                    |
| Operators & maths     | <a href="https://javascript.info/operators">javascript.info/operators</a>                                     | ++/--, ** — mostly familiar              |
| Comparisons           | <a href="https://javascript.info/comparison">javascript.info/comparison</a>                                   | === vs ==, the null >= 0 trap            |
| Logical operators     | <a href="https://javascript.info/logical-operators">javascript.info/logical-operators</a>                     | &&,   , !, short-circuit returns         |
| Nullish coalescing ?? | <a href="https://javascript.info/nullish-coalescing-operator">javascript.info/nullish-coalescing-operator</a> | No Python equivalent                     |
| if / ternary ?        | <a href="https://javascript.info/ifelse">javascript.info/ifelse</a>                                           | cond ? a : b not a if cond else b        |
| Loops                 | <a href="https://javascript.info/while-for">javascript.info/while-for</a>                                     | for, while, break, continue              |
| switch                | <a href="https://javascript.info/switch">javascript.info/switch</a>                                           | Uses === internally — don't forget break |
| Functions             | <a href="https://javascript.info/function-basics">javascript.info/function-basics</a>                         | Parameters, defaults, return             |
| Function expressions  | <a href="https://javascript.info/function-expressions">javascript.info/function-expressions</a>               | Functions as values — important in JS    |
| ■ Arrow functions     | <a href="https://javascript.info/arrow-functions-basics">javascript.info/arrow-functions-basics</a>           | => syntax intro — richer than lambda     |

*Milestone: Solve all exercises at the bottom of each page. They're short and check real understanding.*

### BLOCK 2 Objects

3 days

| Topic                       | javascript.info URL                                                                           | Python Dev Note                            |
|-----------------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------|
| Objects                     | <a href="https://javascript.info/object">javascript.info/object</a>                           | Like Python dicts but with method syntax   |
| Object references & copying | <a href="https://javascript.info/object-copy">javascript.info/object-copy</a>                 | Passed by reference — same as Python dicts |
| ■ Object methods & this     | <a href="https://javascript.info/object-methods">javascript.info/object-methods</a>           | this is the tricky one — read carefully    |
| Constructor & new           | <a href="https://javascript.info/constructor-new">javascript.info/constructor-new</a>         | Like __init__ but different                |
| Optional chaining ?.        | <a href="https://javascript.info/optional-chaining">javascript.info/optional-chaining</a>     | obj?.prop?.nested — very useful            |
| Object to primitive         | <a href="https://javascript.info/object-to-primitive">javascript.info/object-to-primitive</a> | How JS converts objects in expressions     |

*Milestone: Create a user object with methods: login(), logout(), greet(). Experiment with this in each.*

### BLOCK 3 Data Types in Depth

4 days

| Topic           | javascript.info URL                                                                                       | Python Dev Note                                                                                       |
|-----------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Numbers         | <a href="https://javascript.info/number">javascript.info/number</a>                                       | <code>Number.isNaN()</code> , <code>toFixed()</code> , <code>parseInt()</code>                        |
| Strings         | <a href="https://javascript.info/string">javascript.info/string</a>                                       | Template literals, <code>.slice()</code> , <code>.includes()</code>                                   |
| Arrays          | <a href="https://javascript.info/array">javascript.info/array</a>                                         | Like Python lists                                                                                     |
| ■ Array methods | <a href="https://javascript.info/array-methods">javascript.info/array-methods</a>                         | Most important page: <code>map</code> , <code>filter</code> , <code>reduce</code> , <code>find</code> |
| Map and Set     | <a href="https://javascript.info/map-set">javascript.info/map-set</a>                                     | Same concept as Python dict and set                                                                   |
| ■ Destructuring | <a href="https://javascript.info/destructuring-as-signment">javascript.info/destructuring-as-signment</a> | Array and object unpacking — ES6 key feature                                                          |
| JSON            | <a href="https://javascript.info/json">javascript.info/json</a>                                           | <code>JSON.stringify()</code> / <code>JSON.parse()</code>                                             |

*Milestone: Take a raw array of user objects and use filter, map, and reduce to transform it — no loops allowed.*

### BLOCK 4 Advanced Functions

4 days

| Topic                       | javascript.info URL                                                                                 | Python Dev Note                                   |
|-----------------------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------|
| Rest params & spread ...    | <a href="https://javascript.info/rest-parameters-spread">javascript.info/rest-parameters-spread</a> | Like <code>*args</code> but also works on objects |
| ■ Closures & scope          | <a href="https://javascript.info/closure">javascript.info/closure</a>                               | Critical — JS uses closures everywhere            |
| The old var                 | <a href="https://javascript.info/var">javascript.info/var</a>                                       | Read once, then never use <code>var</code> again  |
| setTimeout / setInterval    | <a href="https://javascript.info/settimeout-setinterval">javascript.info/settimeout-setinterval</a> | Scheduling — no Python equivalent                 |
| ■ Arrow functions revisited | <a href="https://javascript.info/arrow-functions">javascript.info/arrow-functions</a>               | How this works in arrows vs regular functions     |

*Milestone: Write a `makeCounter()` function that returns a counter using closure — no classes.*

### BLOCK 5 Classes & Prototypes

3 days

| Topic                       | javascript.info URL                                                                                                             | Python Dev Note                                              |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| Prototypal inheritance      | <a href="https://javascript.info/prototype-inheritance">javascript.info/prototype-inheritance</a>                               | The underlying mechanism — read for understanding            |
| Class basic syntax          | <a href="https://javascript.info/class">javascript.info/class</a>                                                               | Familiar from Python                                         |
| Class inheritance           | <a href="https://javascript.info/class-inheritance">javascript.info/class-inheritance</a>                                       | <code>extends</code> and <code>super</code> — same as Python |
| Static properties & methods | <a href="https://javascript.info/static-properties-methods">javascript.info/static-properties-methods</a>                       | Same as Python <code>@staticmethod</code>                    |
| Private fields #            | <a href="https://javascript.info/private-protected-properties-methods">javascript.info/private-protected-properties-methods</a> | <code>#field</code> syntax — newer JS                        |
| instanceof                  | <a href="https://javascript.info/instanceof">javascript.info/instanceof</a>                                                     | Like Python <code>isinstance()</code>                        |

*Milestone: Model a class hierarchy: `Animal` -> `Dog` -> `GuideDog` with overridden methods.*

**BLOCK 6 Async JS**

4 days

| Topic                   | javascript.info URL                                                                                 | Python Dev Note                                 |
|-------------------------|-----------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Callbacks               | <a href="https://javascript.info/callbacks">javascript.info/callbacks</a>                           | The old way — understand to appreciate Promises |
| ■ Promises              | <a href="https://javascript.info/promise-basics">javascript.info/promise-basics</a>                 | Core async concept                              |
| Promise chaining        | <a href="https://javascript.info/promise-chaining">javascript.info/promise-chaining</a>             | .then().then().catch() pattern                  |
| ■ Promise API           | <a href="https://javascript.info/promise-api">javascript.info/promise-api</a>                       | Promise.all, Promise.race, Promise.allSettled   |
| ■ async/await           | <a href="https://javascript.info/async-await">javascript.info/async-await</a>                       | The modern way — similar feel to Python asyncio |
| Error handling in async | <a href="https://javascript.info/promise-error-handling">javascript.info/promise-error-handling</a> | try/catch inside async functions                |

Milestone: Fetch from [jsonplaceholder.typicode.com/users](https://jsonplaceholder.typicode.com/users) using async/await, handle errors, log results.

**BLOCK 7 Modules**

1 day

| Topic           | javascript.info URL                                                                                   | Python Dev Note                          |
|-----------------|-------------------------------------------------------------------------------------------------------|------------------------------------------|
| Modules intro   | <a href="https://javascript.info/modules-intro">javascript.info/modules-intro</a>                     | Like Python module system                |
| Export & Import | <a href="https://javascript.info/import-export">javascript.info/import-export</a>                     | Named exports, default exports, import * |
| Dynamic imports | <a href="https://javascript.info/modules-dynamic-imports">javascript.info/modules-dynamic-imports</a> | Lazy loading — no Python equivalent      |

Milestone: Split your counter/class code across multiple files and import them correctly using ES modules.

**ES6+ Features — Where to Find Them**

| Feature                  | Block | Link                                                                                                                            |
|--------------------------|-------|---------------------------------------------------------------------------------------------------------------------------------|
| let / const              | 1     | <a href="https://javascript.info/variables">javascript.info/variables</a>                                                       |
| Arrow functions =>       | 1     | <a href="https://javascript.info/arrow-functions-basics">javascript.info/arrow-functions-basics</a>                             |
| Template literals `\${}` | 1 / 3 | <a href="https://javascript.info/string">javascript.info/string</a>                                                             |
| Destructuring            | 3     | <a href="https://javascript.info/destructuring-assignment">javascript.info/destructuring-assignment</a>                         |
| Spread / Rest ...        | 4     | <a href="https://javascript.info/rest-parameters-spread">javascript.info/rest-parameters-spread</a>                             |
| Map / Set                | 3     | <a href="https://javascript.info/map-set">javascript.info/map-set</a>                                                           |
| Optional chaining ?.     | 2     | <a href="https://javascript.info/optional-chaining">javascript.info/optional-chaining</a>                                       |
| Nullish coalescing ??    | 1     | <a href="https://javascript.info/nullish-coalescing-operator">javascript.info/nullish-coalescing-operator</a>                   |
| Classes                  | 5     | <a href="https://javascript.info/class">javascript.info/class</a>                                                               |
| Private fields #         | 5     | <a href="https://javascript.info/private-protected-properties-methods">javascript.info/private-protected-properties-methods</a> |
| Promises                 | 6     | <a href="https://javascript.info/promise-basics">javascript.info/promise-basics</a>                                             |
| async / await            | 6     | <a href="https://javascript.info/async-await">javascript.info/async-await</a>                                                   |
| import / export          | 7     | <a href="https://javascript.info/import-export">javascript.info/import-export</a>                                               |

# Python to JavaScript Cheat Sheet

| Python                             | JavaScript                                       | Notes                           |
|------------------------------------|--------------------------------------------------|---------------------------------|
| <code>def foo():</code>            | <code>function foo() {}</code>                   | Named function declaration      |
| <code>x = lambda n: n*2</code>     | <code>const x = n =&gt; n * 2</code>             | Arrow fn — richer than lambda   |
| <code>[x for x in lst]</code>      | <code>lst.map(x =&gt; ...)</code>                | Transform every item            |
| <code>[x for x in lst if c]</code> | <code>lst.filter(x =&gt; c)</code>               | Keep items matching c           |
| <code>sum(x*2 for x in lst)</code> | <code>lst.reduce((a,x)=&gt;a+x*2, 0)</code>      | Accumulate a value              |
| <code>{'a': 1}</code>              | <code>{ a: 1 }</code>                            | Dictionary / Object literal     |
| <code>None</code>                  | <code>null</code> (also <code>undefined</code> ) | JS has two nothing values       |
| <code>print(x)</code>              | <code>console.log(x)</code>                      | Standard output                 |
| <code>f"hello {name}"</code>       | <code>`hello \${name}`</code>                    | String interpolation            |
| <code>import asyncio</code>        | <code>Promises / async-await</code>              | Async is built-in to JS         |
| <code>class Foo:</code>            | <code>class Foo { }</code>                       | Syntactic sugar over prototypes |
| <code>__init__(self)</code>        | <code>constructor()</code>                       | Instance initialiser            |
| <code>isinstance(x, Foo)</code>    | <code>x instanceof Foo</code>                    | Type checking                   |
| <code>*args</code>                 | <code>...rest</code>                             | Variadic parameters             |
| <code>global x</code>              | (just assign — no keyword)                       | JS walks scope chain outward    |

## Common Pitfalls for Python Developers

|                               |                                                                                                                                           |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>== vs ===</b>              | Always use <code>===</code> . The <code>==</code> operator coerces types — e.g. <code>0 == false</code> is true.                          |
| <b>var hoisting</b>           | <code>var</code> is hoisted to function scope. Use <code>const</code> and <code>let</code> exclusively.                                   |
| <b>this in callbacks</b>      | Regular functions rebind <code>this</code> ; arrow functions capture it lexically.                                                        |
| <b>null vs undefined</b>      | JS has both. <code>null</code> is set by you; <code>undefined</code> means never assigned.                                                |
| <b>Async surprises</b>        | Forgetting <code>await</code> returns a Promise object, not the value — very common bug.                                                  |
| <b>Scope &amp; assignment</b> | Assigning without <code>let/const</code> modifies an outer variable. Python creates a local. Always declare with <code>let/const</code> . |
| <b>Mutable objects</b>        | Objects and arrays are passed by reference, just like Python dicts.                                                                       |
| <b>NaN comparisons</b>        | <code>NaN !== NaN</code> . Use <code>Number.isNaN(x)</code> to check, not <code>x === NaN</code> .                                        |

## Essential Resources

| Resource          | Best For                                                 | URL                                                                                     |
|-------------------|----------------------------------------------------------|-----------------------------------------------------------------------------------------|
| javascript.info   | Primary textbook — free, structured, covers all 7 blocks | <a href="https://javascript.info">https://javascript.info</a>                           |
| MDN Web Docs      | Reference for every DOM method and Web API               | <a href="https://developer.mozilla.org">https://developer.mozilla.org</a>               |
| react.dev         | Official React docs with interactive exercises           | <a href="https://react.dev">https://react.dev</a>                                       |
| Node.js           | Run JS outside the browser while learning Phase 1        | <a href="https://nodejs.org">https://nodejs.org</a>                                     |
| Vite              | Fastest way to scaffold a React or Vanilla JS app        | <a href="https://vitejs.dev">https://vitejs.dev</a>                                     |
| GitHub Pages      | Free static hosting for your project milestones          | <a href="https://pages.github.com">https://pages.github.com</a>                         |
| ESLint + Prettier | Linting and formatting — set up from day one             | <a href="https://eslint.org">https://eslint.org</a>                                     |
| jsonplaceholder   | Free fake API for practising async/fetch in Block 6      | <a href="https://jsonplaceholder.typicode.com">https://jsonplaceholder.typicode.com</a> |

## Tips for Python Developers Specifically

Your algorithmic thinking transfers directly — loops, conditionals, functions all map cleanly.

Learn the browser DevTools console first — it is your REPL and debugger combined.

Don't rush to a framework. The 7 blocks in Phase 1 save months of framework confusion later.

TypeScript is worth learning after Phase 1 — it adds Python-style type hints to JS.

Use Node.js (v20+) locally so you can run JS files from the terminal like Python scripts.

Read error messages carefully — JS errors in the browser console are descriptive and actionable.

Blocks 4 (closures) and 6 (async) deserve the most time — that is where Python devs get tripped up.

*Your Python foundation is a genuine advantage. Work through all 7 blocks, complete every milestone, and the rest of the phases will fall into place.*